Trust Juggler — Final Build Plan

Oct 17, 2026 · @Deep

Why This Is a Safety Problem, Not Just a Fraud Problem

A delivery or ride platform verifies a worker once — ID, background check, face match — then trusts that approval for the life of the account. When the account quietly changes hands (rented, sold, or handed to a friend for a shift), everyone downstream is interacting with a person the platform never checked. That is not a billing problem. It is a physical-safety and legal-liability problem, and it compounds the longer it goes undetected.

What actually goes wrong:

Who is exposed	What can happen	Why it's worse than it sounds
Customers / passengers	An unverified, non-background-checked stranger enters a home, handles food, or drives a passenger, under an identity the platform has already vouched for	The platform's trust signal ("verified driver") is now false, but customers have no way to know that
The platform	A criminal incident involving a rented account becomes a liability and regulatory event traced back to the platform's own verification claims	"We verified this person" stops being true the moment the account was handed off, undermining the platform's legal defense
The original verified worker	Their verified identity, face, and rating are now being used by someone else's conduct — a bad delivery, an accident, a complaint — damages a reputation and account they still legally own	They bear consequences (suspension, legal exposure) for actions they didn't take
Other workers	Rented accounts undercut fair competition for shifts/zones and erode trust in the worker pool as a whole	A platform known to tolerate account-sharing loses legitimacy with both regulators and honest workers
Insurers	Trip/ride insurance is underwritten against the verified individual's risk profile, not an unknown substitute	Claims processes and liability coverage assume the verified person was actually driving

Why the obvious fixes fail: re-verifying identity on every login (face scan at every shift start) adds friction honest workers won't tolerate and still misses mid-shift handoffs. One-time verification at signup is cheap but trusts a single moment forever. Trust Juggler's bet is that *behavior* — how someone moves, rides, and taps — is a continuous signal that neither extreme uses, and it can catch a handoff within the session it happens in, not weeks later after a complaint.

Executive Summary

Trust Juggler asks one question per session: **has the shape of this account's behavior changed in a way life doesn't explain?** Instead of re-verifying identity on every login (locks out honest riders) or trusting one onboarding check forever (lets rented accounts run for weeks), the system scores every session on two independent axes — **context continuity** (device, city, network) and **behavioral continuity** (motion, touch, operational rhythm) — and only escalates the sessions whose combination looks like a handoff.

Why this over Receipt Graveyard: Receipt Graveyard rewards API-calling (OCR → rules → verdict) and is easy to clone. Trust Juggler rewards an actual decision architecture: a two-axis trust model, dual-timescale baselines, split-rental clustering, and a calibrated false-challenge rate — none of it reduces to "call GPT-4o and parse JSON." It is harder to build in 10 hours, which is exactly why it stands out.

One-line pitch: *"We don't ask 'who are you' on every login — we ask 'has this account's behavior changed in a way life doesn't explain,' and we promise honest riders a measured, low false-challenge rate."*

What "done" looks like in 10 hours: a live demo where a rider logs in normally, hands the phone to someone else mid-shift, and the dashboard's trust score visibly drops and triggers a context-appropriate, non-disruptive challenge — while a *second* simulated rider who just got a new phone sails through untouched.

Productization: Selling This as Enterprise Software

Business model: B2B SaaS sold to delivery/ride platforms as a drop-in trust layer, not a consumer product. Two integration paths:

1. **SDK-first (primary):** a lightweight mobile SDK embedded in the platform's own rider/driver app, collecting motion and touch telemetry on-device and sending only derived features or scores to our API — never raw sensor streams. This is the privacy-by-design pitch and the realistic integration path for a real platform.
2. **API-only (fallback for platforms that can't ship an SDK quickly):** the platform forwards whatever it already logs — GPS pings, event timestamps, device/IP metadata — to our ingest API; we score on operational and GPS signals alone, with reduced accuracy since touch/motion detail is unavailable.

Pricing shape (illustrative, not quoted to customers without real unit-economics modeling): per-active-worker-per-month, tiered by signal depth (operational-only vs. full SDK), the way fraud-prevention vendors in this space (Section below) typically price.

What a 10-hour hackathon prototype should actually show — scope it down hard, don't try to build the SDK:

- A **simulated integration**: a small "customer platform" demo app sending synthetic session events to our ingest API, proving the API contract works end-to-end.
- The **admin dashboard** (see System Design below) as the real deliverable — this is what a judge or a prospective enterprise buyer actually evaluates, because it's the part they'd use daily.
- The **red-team simulator** doubling as a sales demo: "here is what a handoff looks like to us, and here is what a legitimate life change looks like, and here is why we don't confuse them."
- Do **not** promise or demo real on-device SDK data collection in 10 hours — say explicitly in the pitch that the SDK is the engineering roadmap, not what's running tonight.

Full-roadmap scope (post-hackathon, state this explicitly as roadmap, not built): native iOS/Android SDK, SOC 2 compliance program, per-region data residency (relevant under India's DPDP Act and similar laws elsewhere), a webhook/alerting integration for the customer's own ops tooling, and a self-serve onboarding flow.

Reference Sites: Who to Model the Selling Website After

Real companies in adjacent spaces (behavioral biometrics, fraud detection, identity verification) — useful for studying site structure and positioning, not for copying claims. Verify current URLs and messaging before citing numbers from any of them.

Company	Space	Why it's a useful reference
BioCatch	Behavioral biometrics for fraud/account-takeover	Closest direct analogue — continuous behavioral authentication, enterprise-sold, worth studying for how they explain "invisible" authentication to a non-technical buyer
Persona	Identity verification platform	Clean, developer-first SaaS site structure (docs-first homepage, integration-focused)

Incognia	Location/device-based fraud signals for mobile apps, used by ride-hailing and delivery platforms	Same buyer persona (gig/delivery platforms), useful for seeing how they pitch to that exact audience
Sardine	Fraud and risk scoring using behavior + device signals	Good example of a quadrant/risk-score dashboard presented to a non-technical ops team
Castle	Account-takeover and bot detection	Developer-facing API docs + a simple risk-score UI, close to what the admin dashboard should look like
Arkose Labs	Bot/fraud mitigation	Useful for "challenge, don't block" messaging — the friction-aware verification pitch

Site structure to borrow: homepage leads with the risk ("account takeover costs gig platforms X"), not the model; a product section shows the dashboard screenshot, not architecture diagrams; a docs/API section is separate from the marketing site; case studies or an interactive demo sandbox close the page. Don't fabricate a customer logo wall or quoted results for a prototype — use "request a demo" instead of named case studies until real ones exist.

System Architecture

[embed: node/eccd3444-28c3]

Each scorer answers a narrower question (operational rhythm, riding motion, device/network context) so no single spoofed signal can fake all three at once. The combiner never outputs a single confidence number — it outputs a quadrant.

System Design: Admin Dashboard + Driver App

[embed: node/fcd99791-9723]

Driver mobile app: the platform's existing rider/driver app with our SDK embedded. It never sends raw GPS traces or raw touch recordings to the backend — it extracts features on-device (riding-style vectors, tap-rhythm vectors) and sends those plus coarse GPS, minimizing both bandwidth and privacy exposure.

Admin web dashboard (the actual prototype deliverable): gives platform ops teams fleet-wide visibility:

- **Fleet quadrant view** — every active driver plotted on the context-continuity × behavior-continuity axes, so anomalies are visually obvious at a glance.

- **Per-driver trust timeline** — drill into one driver, see the score history and every event that moved it.
- **Review queue** — prioritized by risk, each case pre-loaded with its evidence summary (reason vector) so a human reviewer doesn't start from zero.
- **Cohort fairness panel** — false-challenge rate broken out by cohort (new phone, city move, appearance change), so ops can audit the system for bias, not just effectiveness.
- **Red-team / scenario simulator** — doubles as both an internal test harness and a sales demo.

Signal Taxonomy

Mapped against the behavioral-biometrics survey literature (140+ studies: 28 motion, 19 gait, 20 keystroke, 29 touch gesture, 16 voice, 34 multimodal). Voice is dropped — riders don't speak into the app.

Tier	Signal	Delivery-rider equivalent	Relevance	Build effort (10h)
1 — Operational	Acceptance rate, response time, shift timing	Already logged by the platform API, no SDK needed	High	Trivial — just feature engineering
1 — Operational	Preferred zones, route efficiency, idle time	Already logged	High	Trivial
2 — Motion	Gait (walking pattern)	Riding style: acceleration from stops, braking, cornering G-force	High	Simulate from GPS + H-MOG priors
2 — Motion	Accelerometer / gyroscope	Phone hold angle, vibration profile while riding	High	Simulate from H-MOG's motion sessions
3 — Touch	Keystroke dynamics	Tap rhythm on "Accept Order" (riders don't type much)	Low-direct but real	Needs mobile SDK — simulate for demo

3 — Touch	Touch gesture (swipe, pressure, area)	Same phone, different grip/thumb — the direct "different person" detector	High	Needs mobile SDK — use H-MOG touch events, simulate
—	Voice	N/A	None	Skip entirely
Combined	Multimodal fusion	All of the above combined	Essential — 34-study literature says multimodal always outperforms any single signal	This is the architecture, not an add-on

H-MOG dataset relevance: H-MOG captures accelerometer, gyroscope, magnetometer and touch events (tap/swipe/scroll, with area, pressure, coordinates, duration) while 100 users sit, walk and stand. Its *walking* state is the closest proxy for a rider handling a phone at a stop; its tap tasks map directly to accepting/confirming orders. Gap: H-MOG users are indoors, not on a vibrating bike — apply a vibration filter in feature engineering and use H-MOG only to validate inter-person variance, not as a literal riding dataset.

ML Pipeline in Depth: Signals, Datasets, APIs, Pretrained Models

Per-signal breakdown

Signal	What it captures	Dataset to prototype with	Model	Output
GPS / mobility	Riding-style signature: acceleration from stops, braking, cornering, route efficiency, zone/time patterns	ODIDMobTraj (delivery-rider trajectories), GeoLife (baseline), NUMOSIM (synthetic anomaly injection for evaluation)	IsolationForest or LSTM autoencoder on trajectory-derived features (speed/accel histograms, turn-rate distributions)	Per-session anomaly score
Touch sensor	Tap rhythm, swipe velocity, touch area/pressure,	H-MOG (tap/swipe/scroll events with area, pressure, coordinates, duration)	One-class classifier or autoencoder fine-tuned per	Per-event anomaly score

	inter-tap timing — the direct "same phone, different grip" signal		rider on H-MOG-pretrained touch-embedding features	
Motion (accel/gyro)	Riding vibration and hold-angle signature	H-MOG (accelerometer/gyroscope while walking/sitting, adapted with a vibration filter)	LSTM autoencoder, architecture from public time-series-anomaly examples, trained on H-MOG + synthetic rider sim	Per-session anomaly score
Operational	Acceptance rate, response latency, shift timing, zone preference	The platform's own historical logs (synthesized for the prototype)	IsolationForest (PyOD)	Per-session anomaly score
Pattern detection (fusion)	Combines all of the above into context vs. behavior continuity	N/A — this is the combiner logic, not a trained model	Rule-based combiner + a calibrated threshold (conformal prediction on each rider's own history)	Quadrant + trust score

Pipeline

Ingestion → per-signal feature extraction (on-device for touch/motion, server-side for GPS/operational) → per-signal anomaly scorer → two-axis combiner → conformal threshold check → decision (continue / lightweight challenge / step-up verification / human review) → GPT-4o/Claude call generates the plain-language reason vector only for sessions that cross the challenge threshold, to control LLM cost.

APIs

Endpoint	Direction	Purpose

POST /v1/sessions/ingest	Driver app → backend	Submit a session's feature vector (not raw sensor data)
GET /v1/drivers/{id}/score	Admin dashboard → backend	Current trust score + quadrant for one driver
GET /v1/fleet/scores	Admin dashboard → backend	Fleet-wide quadrant snapshot
POST /v1/drivers/{id}/declare-change	Driver app → backend	Rider-declared context change (new phone, moved city) — the fast lane
POST /v1/webhooks/alert	Backend → customer's own ops tooling	Push a high-risk escalation out, instead of requiring polling

Throughput note: no production load figures are quoted here — target requests-per-second depends entirely on the customer's active-driver count, which isn't known for a hackathon prototype. Design the ingest endpoint to batch session events (one call per session, not per sensor sample) so the realistic load is sessions/second, not raw sensor-sample rate; load-test against an assumed number once a real customer's fleet size is known, rather than quoting a number now.

Pretrained models — what's genuinely available off the shelf vs. what must be trained

Component	Genuinely pretrained & public?	Reality
GPT-4o / Claude (reason-vector explanations)	Yes	Real, call via API, use as-is
Face embedding models (FaceNet, ArcFace)	Yes	Real, public pretrained weights exist, usable for the optional selfie step-up check
Human Activity Recognition (HAR) models pretrained on UCI-HAR / WISDM	Yes, for generic activity classes (walking/sitting/standing)	Real public datasets and reference models exist, but they classify *activity type*, not *identity* — useful as a feature extractor backbone, not a drop-in rider-identity model

A "rider trust" or "driver identity" pretrained model	**No**	Does not publicly exist. This is the core training-from-scratch work of the project — don't claim or imply a pretrained model solves this; H-MOG/GeoLife/ODIDMobTraj are training data, not pretrained weights
IsolationForest / One-Class SVM anomaly detectors	N/A (not a pretrained-weights kind of model)	Trained fresh per rider or per cohort from day one — that's inherent to how they work
LSTM autoencoder for motion anomaly	Architecture is public (reference implementations exist); weights are not	Train the architecture on H-MOG + synthetic data; there is no public pretrained checkpoint for this exact task

Be explicit about this split in any pitch or paper: claiming a "pretrained rider-identity model" where none exists is the kind of fabricated-capability claim that collapses under a technical judge's first question.

Top Differentiators (ranked, deduplicated)

1. **Two-axis trust: Context Continuity × Behavioral Continuity.** Not one score — a quadrant. Context changed + behavior stable = legit life change, re-baseline silently. Context stable + behavior step-change = handoff signature, escalate. Both changed = ambiguous, lightweight challenge. Neither changed = nothing. This single diagram explains the whole system and maps directly onto the problem statement's "same city, same phone, different person" line.
2. **Dual-timescale baselines (defeats the boiling-frog attack).** Keep a fast baseline that adapts to normal drift AND a frozen identity anchor set at the last strong verification. Legit drift is tolerated; cumulative distance from the anchor eventually forces a challenge, which resets the anchor. Directly answers "what if the attacker adapts slowly?"
3. **Split-rental detection.** Step-change detectors miss accounts shared between two people who alternate shifts. Cluster each account's sessions (mixture model) and flag accounts showing two stable behavioral identities, especially when clusters correlate with time of day.
4. **Conformal-calibrated honest-worker guarantee.** Calibrate each rider's challenge threshold on their own history so you can state "at most ~1% of honest

sessions trigger a challenge," broken out by cohort (new phone, city move, new rider). Turns "we try not to block people" into a measured, defensible promise.

5. **Cost-aware challenge timing.** Don't just threshold — decide whether AND when to challenge by comparing P(takeover) × harm against friction cost. Never challenge mid-ride; prefer natural pauses (after delivery, before payout); challenge earlier when stakes are high.
6. **Learned legit-change signatures.** Instead of hand-written rules ("new phone is fine"), learn from the whole rider population what a normal behavioral wobble looks like after each life event (phone change, city move), then score an individual rider's change against that population expectation rather than only their own history.
7. **Rider-declared context change (fast lane).** Let riders say "I got a new phone" before it happens. A declared change gets a quick check instead of silent suspicion — cooperative friction instead of opaque deactivation.
8. **Explainable reason vectors.** Every escalation shows which signals moved and what would restore trust ("we couldn't confirm it's you after your pattern changed; a quick check will fix this") — counters the opaque-deactivation complaints common in real gig-platform fraud research.
9. **Account graph layer.** Link accounts sharing payout destinations, devices, or faces to catch rental rings and banned workers returning under new IDs — covers the cold-start period where behavioral signal alone is too thin.
10. **Privacy-by-design.** On-device behavioral embeddings (send a score, not raw touch recordings); numerical features only, never raw keystroke content; auditable reason for every verification decision.

Edge Case Matrix

Deduplicated across all sources. Priority: P0 = must handle in the demo, P1 = strong differentiator if time allows, P2 = name it in the pitch, don't build it.

A. Honest riders who look suspicious (false-positive traps)

#	Edge case	Why it breaks a naive system	Expected handling	Priority
1	Same phone, different person	Device ID, IP, location all unchanged	Behavioral step-change alone must be strong enough to flag	P0

2	Same rider, new delivery zone / city move	Location change alone looks like takeover	Location alone never accuses; check behavior continuity	P0
3	Replacement phone / reinstalled app	New device ID looks unfamiliar	New device ≠ new person; verify, don't assume	P0
4	Different shift / unusual hours	Schedule deviation flagged as suspicious	Down-weight schedule changes unless corroborated	P0
5	Injury, gloves, rough roads, rushing	Touch/motion noise looks like a new identity	Lower confidence on noisy input, don't conclude identity change	P0
6	Phone change + city move together	Two context signals change at once, looks like takeover	If behavior is stable, treat as one combined context reset	P1
7	Illness, new bike, hand injury	Riding/motor pattern shifts for real reasons	Gradual drift shape + optional rider-declared change	P1
8	Returning after a long break	Stale baseline looks "new"	Decay baseline confidence over inactivity, soft re-warm-up	P1
9	Festival, monsoon, strikes, surge	Everyone's behavior shifts at once	Compare rider to cohort's shift that day, not own history alone	P1
10	Appearance change (beard, glasses)	Selfie check fails for the real person	Failed selfie alone never penalizes; route to alternate/human review	P1
11	Poor lighting / cheap camera	Selfie quality failure	Allow retries, quality guidance, non-selfie fallback	P2
12	New rider, no behavioral history	No baseline exists (cold start)	Cohort priors, wider thresholds, lean on non-behavioral checks	P0

B. Attackers who look legitimate (false-negative traps)

#	Edge case	Why it breaks a naive system	Expected handling	Priority
13	Gradual / slow-drift account handoff	Adaptive baseline absorbs the change	Frozen identity anchor + cumulative distance from it	P0
14	Split rental (owner and renter alternate)	Looks like one noisy person	Cluster sessions into behavioral identities, test for two stable modes	P1
15	Renter copies owner's schedule/zones	Operational signals match	Weight motion/touch higher; payout-account correlation	P1
16	Renter deliberately imitates owner's style	Imitation can defeat single-signal detectors	Combine independent signal types; step up when uncertain	P1
17	Short-term rental (one shift)	Too few sessions to trigger persistence rules	Within-shift change-point detection; fast path for high-stakes events	P2
18	Rented right after a verified selfie	Trust is at its peak, attacker exploits the window	Trust decays over time; re-test after payout/session milestones	P1
19	Selfie-by-proxy (owner passes check, renter rides)	Challenge passes, rider differs after	Tie challenge timing/location to pickup point; compare behavior after pass	P2
20	Spoofing: emulator, GPS faker, scripted taps	Fake behavior can mimic population averages	Separate tamper-detection / bot-pattern layer	P2

C. Data, model, and system problems

#	Edge case	Risk	Expected handling	Priority
21	Weak internet / offline usage	Events arrive late, out of order, or not at all	Mark "insufficient evidence"; never treat missing as suspicious	P0
22	Two riders with naturally similar behavior	Biometrics are probabilistic, not exact	Require corroborating evidence, never a single-signal verdict	P0

23	Model mistakenly learns the attacker's behavior	Baseline update poisoning	Update baseline only after confidence threshold; keep rollback audit trail	P1
24	Sparse riders (few sessions/week)	Never builds a stable baseline	Longer windows, report lower confidence explicitly	P1
25	App update / OS update changes sensor sampling	Looks like population-wide behavior shift	Version-tag features, re-normalize per device/app-version	P1
26	Model/API outage	Risk scoring unavailable	Documented fallback policy; never silently flag everyone	P0
27	Concurrent sessions, same account	Could be sharing or a logging bug	Check for logging artifact before flagging	P2
28	Clock/timezone errors	Fake "impossible" event sequences	Prefer server timestamps	P2

D. Policy, fairness, and operations

#	Concern	Risk	Expected handling	Priority
29	Selfie bias across skin tones	Published research shows worse facial-match accuracy for some groups	Monitor challenge-failure rate by cohort; always keep human-review path	P1
30	Appeal loop	False positive = lost income, no recourse	Clear explanation, alternate verification route, time-bound review	P1
31	Challenge fatigue	Frequent challenges push honest workers off the platform	Cap challenge rate per rider per period; track as a metric	P0
32	Safety during challenge	Challenging mid-ride is dangerous	Never challenge mid-ride; wait for a natural pause	P0
33	Mass-event false-positive spikes	Outage/app update floods the review queue	Circuit breaker that raises thresholds on population-wide shift	P1

34	Data privacy	Behavioral data is sensitive	On-device feature extraction, minimal retention; check India's DPDP Act before claiming compliance	P1

What to build first if time is short

1. Phone change + city move together (proves the context/behavior split works)
2. Slow-drift renter (proves the frozen anchor works)
3. Split rental (proves clustering works)
4. Cold start and missing data (the most common real-world failure)
5. Failed selfie by an honest rider (shows fairness thinking + human-review fallback)

Public Datasets

All publicly available, no licensing cost.

Dataset	Link	What it has	Use in this build
H-MOG	hmog-dataset.github.io/hmog	100 users, accelerometer/gyroscope/magnetometer + touch events (tap, swipe, scroll — area, pressure, coordinates, duration) while sitting/walking/standing	Pretrain the motion/touch anomaly scorer; validate inter-person distinguishability for the pitch
GeoLife	Microsoft Research	GPS trajectories from 182 users over years	Baseline for GPS trajectory anomaly detection
ODIDMobTraj	github.com/Nicholas0027/ODIDMobTraj	Delivery-rider GPS trajectories	Closest real analogue to the rider population; seed the synthetic

			simulator's priors
NUMOSIM	via GeoAnomalies'24	Synthetic mobility anomaly benchmark	Validate detector performance against known injected anomalies

Code / libraries

Resource	Install	Use
PyOD IsolationForest	pip install pyod	Per-rider operational anomaly scorer
keras-io time-series anomaly autoencoder	HuggingFace reference implementation	Base LSTM autoencoder for the motion scorer
movingpandas	pip install movingpandas	GPS trajectory processing
GPT-4o / GPT-4o-mini API	OpenAI	Plain-English reason-vector explanations for escalations

References

Citation	What it gives this project
arXiv:2404.11844 — illegal driver substitution via GPS	Direct prior work on your exact problem: detecting a driver swap via trajectory/behavior
arXiv:2510.12076 (BeSTAD)	Per-rider unsupervised behavioral anomaly detection method
arXiv:2001.08578 — 140+ study survey of behavioral biometrics	Validates the motion / gait / keystroke / touch / voice / multimodal taxonomy used here
arXiv:2203.07300 — mobile behavioral biometrics	Touch + motion combined for continuous authentication

iBAT (HAL-01302049)	Isolation-based trajectory anomaly detection, AUC > 0.99
FraudMove (IJGI, 2021)	Real-time trajectory outlier detection for fraudulent drivers
H-MOG dataset paper — "A Multimodal Data Set for Evaluating Continuous Authentication Performance"	Source and documentation for the H-MOG dataset used to pretrain the motion/touch scorer

Pitch-ready citation sentence: "We validated our signal selection against H-MOG, a 100-user multimodal biometric dataset, which confirms accelerometer patterns and touch dynamics are sufficiently person-distinct to separate users; we adapt this to delivery riders, where the equivalent of gait is riding style and the equivalent of keystroke dynamics is order-acceptance tap rhythm."

No benchmark numbers, accuracy percentages, or results are stated above as if measured — run your own evaluation on the synthetic/simulated sessions before quoting any number in the pitch deck or README.

10-Hour / 3-Person Build Plan (Trust Juggler)

Non-negotiable: freeze the two-axis contract (session schema: device_id, ip_subnet, zone, accel[], gyro[], touch_events[], accept_latency_ms, timestamp) in hour 1 and don't touch it — that's what keeps three people unblocked in parallel.

Hours	ML	Backend	Frontend	Gate at end of block
0–1	Pick H-MOG as the motion/touch prior source; sketch the operational + motion scorer interfaces	Set up repo, API keys, DB schema for sessions and riders	Wireframe the dashboard: session list, trust score, quadrant view	Session JSON schema frozen
1–5	Build operational IsolationForest (PyOD) on synthetic rider features; build motion LSTM autoencoder pretrained on H-MOG, fine-tuned on a synthetic rider	Build the context-change detector (device/IP/zone rules) and the two-axis combiner (context score × behavior score → quadrant); stand up the REST API	Build the live dashboard shell: per-rider trust-score timeline, the quadrant chart (Visualize chart: context continuity vs behavioral	ML scorers return a number on synthetic input; API serves a mock combiner response

	simulator seeded from ODIDMobTraj priors		continuity, one dot per session)	
5–8.5	Wire real scorer output into the API; build the red-team simulator: 4 attacker types — lazy renter, slow-drift renter, split-shift sharer, phone-change+city-move (the honest-rider trap)	Wire ML output → combiner → API end-to-end; add the dual-timescale baseline (fast baseline + frozen anchor) and basic conformal threshold calibration	Wire dashboard to the live API; add the "rent this account" toggle that switches a live session between owner and renter behavior	End-to-end pipeline live: a simulated handoff visibly changes the quadrant in the dashboard
8.5–10	Tune thresholds against the red-team simulator; write the GPT-4o reason-vector explanation call	Bug fixes; seed demo data for all 4 red-team scenarios + 2 honest-rider edge cases (new phone, city move)	Polish UI; build the fairness/false-alarm panel (toggle between scenarios); pitch slides	Demo-ready: every scenario in the red-team + honest-rider set produces the correct quadrant

Division of labor rationale

No role blocks on another past hour 1: ML can develop scorers against synthetic data without the real API; backend can build the combiner against a mocked ML response; frontend can build the dashboard against a mocked API response. All three converge only at the hour-5 integration gate.

Demo Script and Pitch

Live demo sequence (3 minutes):

1. Show a rider's dashboard: trust score steady at ~94, quadrant shows "stable/stable."
2. Click "rent this account" — the same session now runs renter behavior on the same device/IP. Watch the quadrant shift to "stable context / step-change behavior" in real time; trust score drops, system issues a lightweight challenge timed for a natural pause (not mid-ride).
3. Reset, then simulate a phone change + city move together on a different rider. Quadrant shows "context changed / behavior stable" — system re-baselines silently, no challenge. This is the comparison that proves the architecture: same

magnitude of context change, opposite verdict, because behavior tells the real story.

4. Open the red-team panel: run all 4 attacker types + the 2 honest-rider edge cases side by side, show detection time and false-challenge rate per cohort.

Closing pitch line: "We don't ask *who are you* on every login — we ask *has the shape of this account's behavior changed in a way life doesn't explain*, and we can promise honest riders a measured, low false-challenge rate while catching handoffs that a static one-time check would miss for weeks."

If a judge pushes on accuracy numbers: say plainly that thresholds were tuned against the red-team simulator and synthetic data, not production traffic — report what the simulator measured, never a fabricated production accuracy figure.